



UNIVERSITÀ
degli STUDI
di CATANIA

IOT Systems and Technologies

Embedded Systems

davide.patti@dieei.unict.it

What is the Internet of Things



Start with a device (a “thing”)

What is the Internet of Things



Add **computational intelligence** to improve the function of the device



What is the Internet of Things

Add a **network connection** to further enhance its function



Traditional Refrigerator

- Keeps items cold
- Doesn't do much else



Intelligent Refrigerator

Tells you

- When the door remains open
- When the water filter needs replacing
- When you are low on butter
- When you buy food with high fat content
- What recipes match its content

**Enhanced functionality but still
not networked**



IoT Refrigerator

- Orders food items when stock is low
- Searches for lowest food prices
- Orders water filter when needed
- Anticipates your meals; orders food preemptively
- Searches news sites for worldwide food price trends
- Provides consumption information to business for marketing purposes

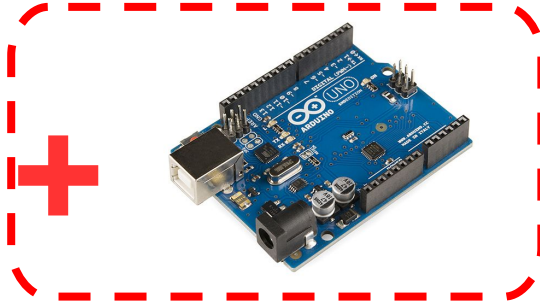


**Greatly enhanced functionality,
Internet required**

Ok... but where Computational
Intelligence comes from?

Embedded System

ES adds to the object Computational Intelligence



Embedded System

- **Microcontroller (MC):** the computational element of the Embedded System
- Interaction with the physical world:
 - **Sensors** (Input: physical world->MC)
 - **Actuators** (Output: MC->physical world)

ES = Microcontroller+Sensors+Actuators



Cos'è un Sistema Embedded?

Il Concetto "Embedded"

L'utente interagisce con l'**oggetto**, non con il computer contenuto al suo interno.

"La mia auto mi ha avvisato di rimettere le mani sul volante..."

"La lavatrice mi ha detto che ha finito il lavaggio..."

"Il mio aspirapolvere è stanco?!?"

Focus Principale

- **Interazione con il Mondo Fisico**
L'intelligenza del sistema agisce direttamente sull'ambiente circostante.
- **Oltre l'Interfaccia Utente-Computer**
NON è un utente umano che interagisce con un'interfaccia computer tradizionale.



Invisibile ma onnipresente

General Purpose vs. Special Purpose

Computers (General Purpose)

Multi-functional & Versatile

Complexity is exposed to the user:

- Installing applications
- Managing & moving files
- Connecting external devices
- Handling various cables (*)

Embedded Systems (Special Purpose)

Specific & Dedicated Task

- Complexity is **hidden** inside the device
- Only core functionality is exposed
- User doesn't perform maintenance tasks (*)
- Adds **specialized intelligence** to objects, shifting interaction focus.

The user interacts with the Object, not the computer within.

(*) refers to operational overhead and technical management tasks common in general-purpose computing.

How ES and IoT are related

IoT: ES Inside an Object

Integrating an Embedded System (ES) into a physical object creates an **IoT device**.

Example: Smart Pen

Uses sensors (pressure, temperature) and actuators (ink flow) to enhance a traditional tool.

ES is not always IoT

An ES can exist independently without being part of a pre-existing traditional object.

- Microcontroller + sensors/actuators
- Dedicated standalone tasks

Example: Monitoring Station

A standalone device measuring humidity and wind speed.



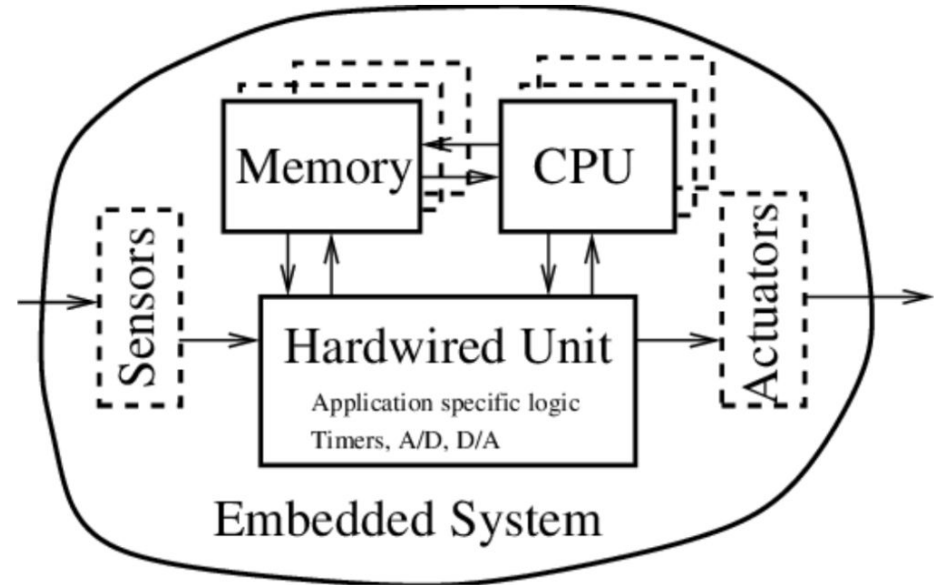
ES “Brain”: The Microcontroller

Embedded System = Microcontroller +Sensors+Actuators



- The MC process data: input data from physical world and output data to the physical world
- **Inputs:** a temperature, a button is pressed, a light is below some level
- **Outputs:** turn on a LED, show something on display, emit a sound

- The core of a Microcontroller consist of a Memory and a CPU (similar to a computer)
- **Memory:** where to store programs to be executed and data to be processed
- **CPU:** Central Processing Unit, takes the instructions from memory and execute them



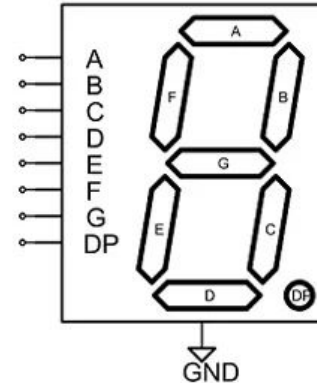
Questions

- What is a microcontroller?
- What is the difference to a microprocessor?
- Why do we need microcontrollers?

Example Heat Control System

How many bit signals are involved from/to the external world?

- Periodically read the temperature:
 - Analog value (16 levels) is digitized by sensor (4-bit)
- Control heating according to the temperature:
 - Turn heater on/off (1 bit)
- Display the current temperature:
 - simple 3-digit numeric display (8+3 bits)
- Allow the user to adjust temperature thresholds:
 - Buttons (4 bits)

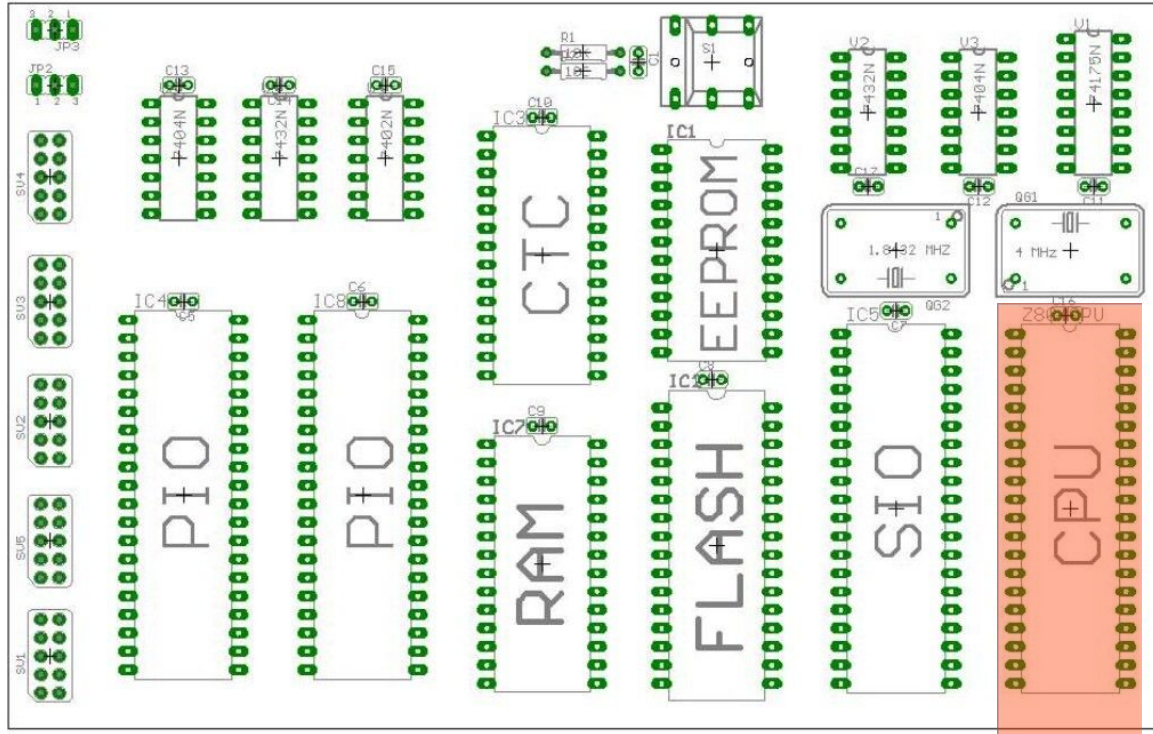


*We need some computing device able to
interact with the external world with 20 bits*

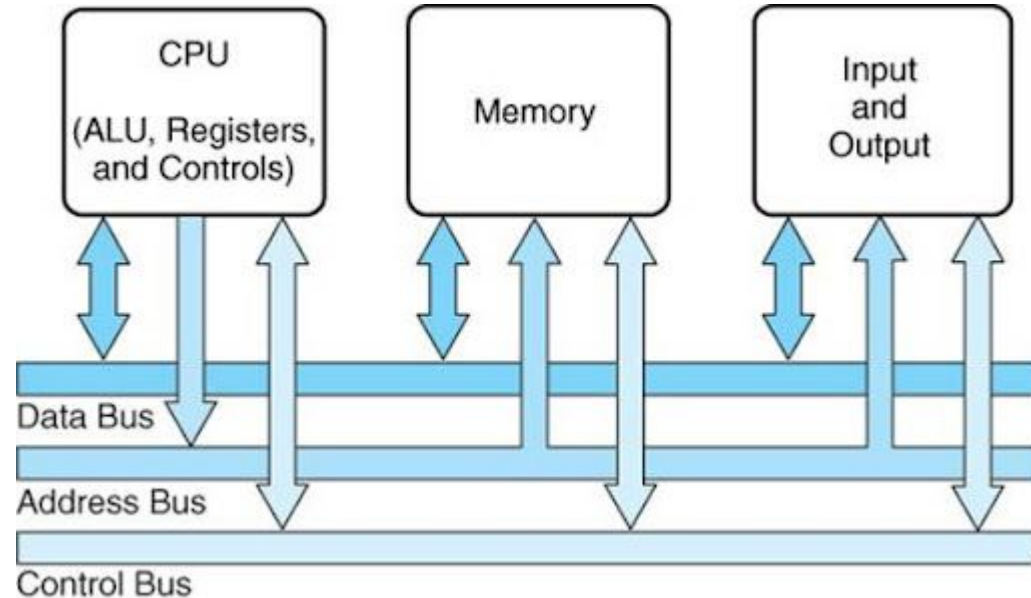
*What's the problem? Just use the usual CPU of
a computer!*

Z80 CPU Based Design

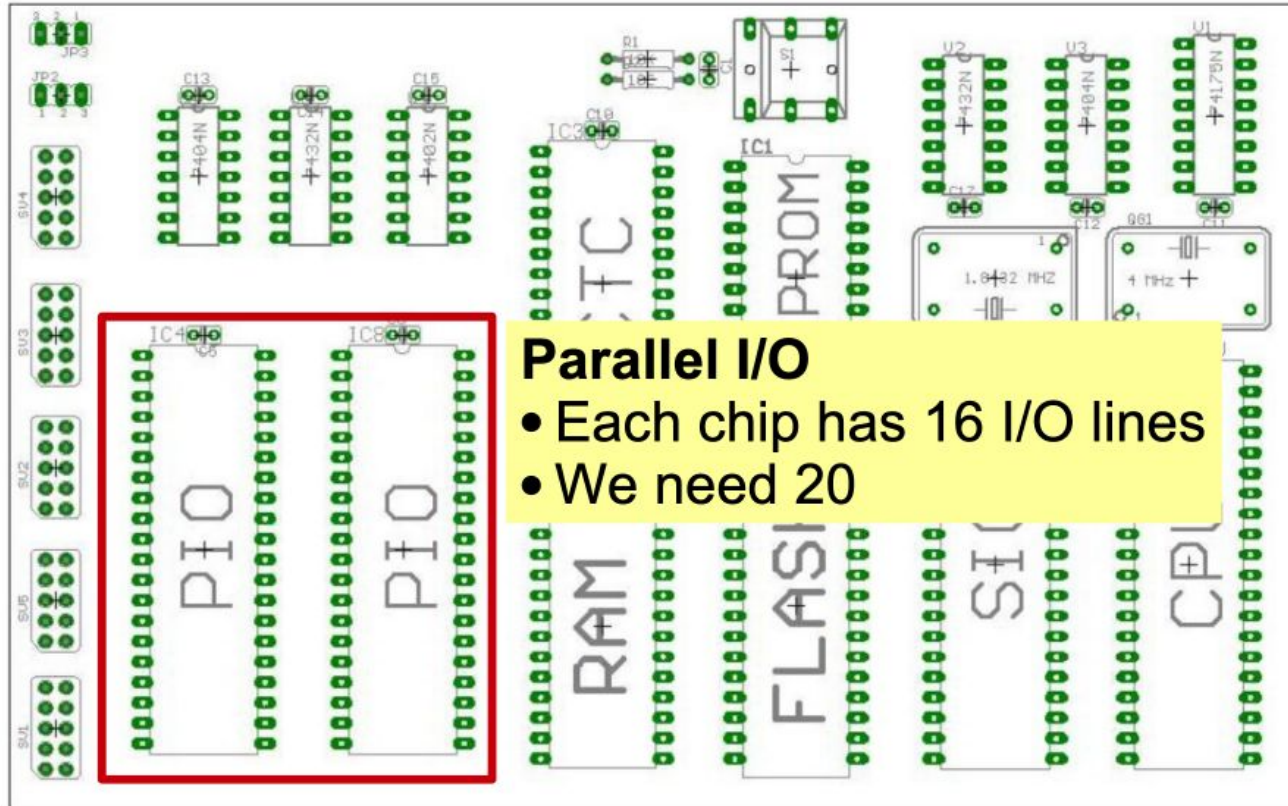
The CPU is only the computational part (ALU, registers)



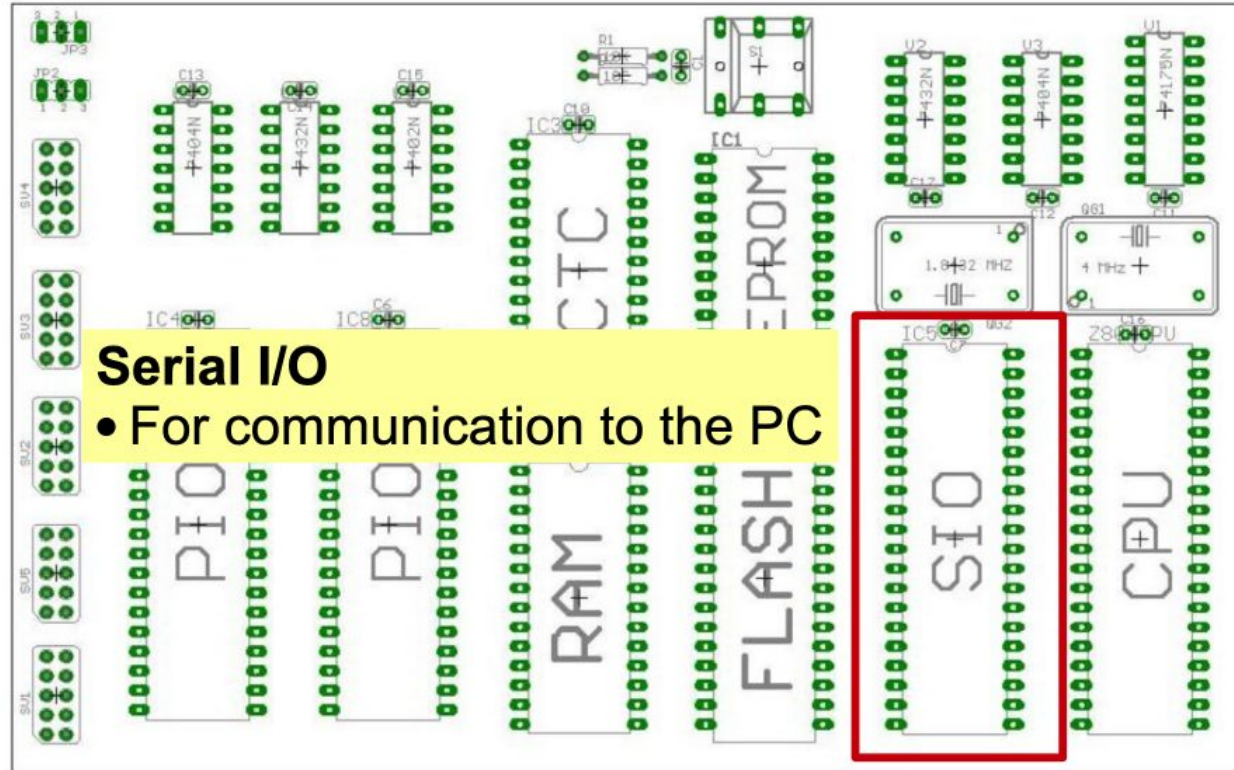
Z80, like any other CPU microprocessors, requires many other external chips to work...



The interface to the world requires some hardware that manage the 20 bits, here we have two chips each one with 16 lines

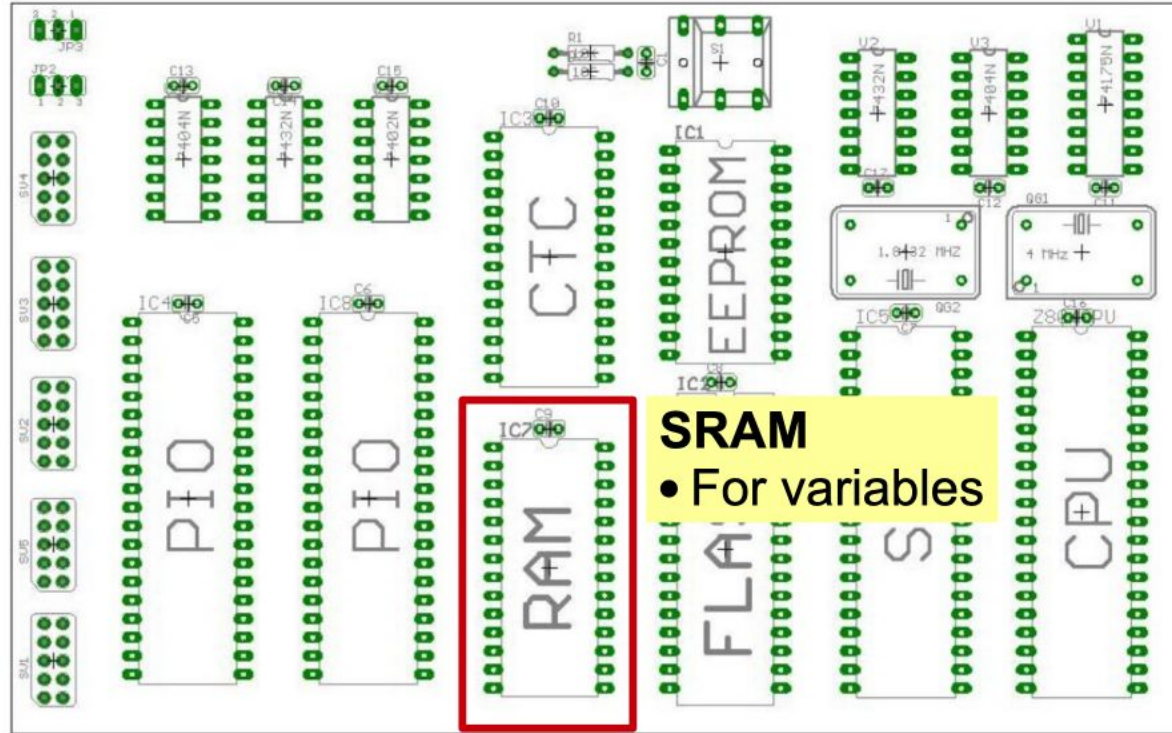


Other chips are dedicated to various purposes, e.g. interfacing with other computers, connecting displays, printers



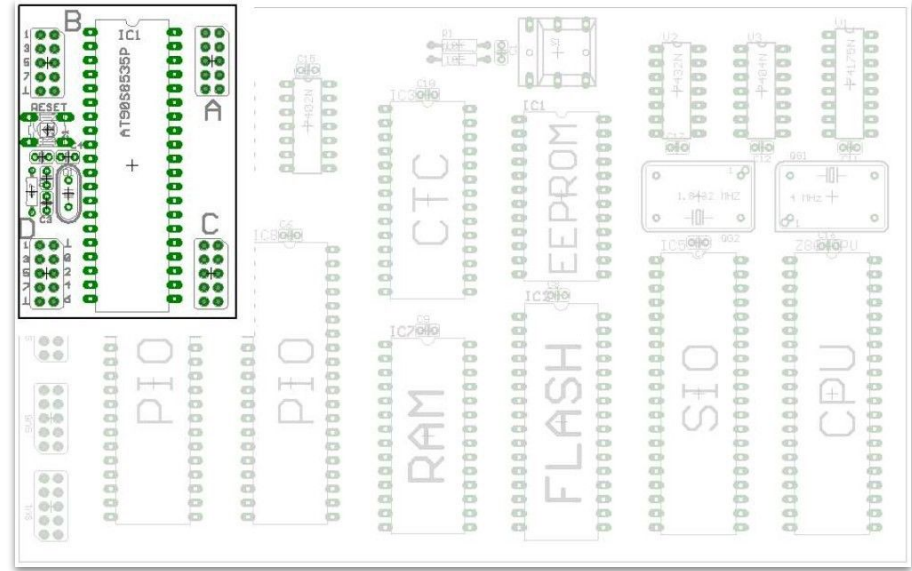
..also, memory chips are present: BIG memory = better because: “*who knows which program will be run in the future?*”

...is that ok in our case ??



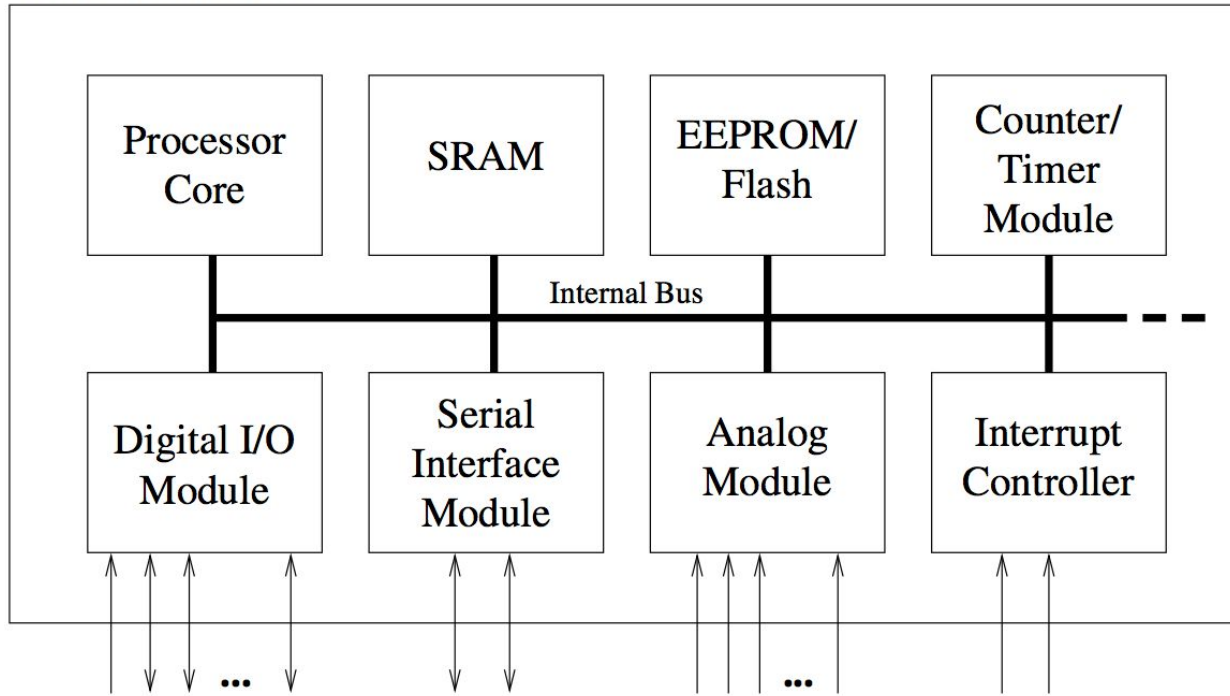
OPPOSITE IDEA: Integrate only a specific subset of what is required into a single chip

- Dramatically reduces PCB size and simplifies the layout
- A single chip reduces the total number of components and manufacturing costs
- A Z80 board might require separate RAM, ROM, ADC, timers, and I/O controllers, making the design larger, more power-hungry, and complex.



The same task can be achieved with an microcontroller, which integrates these features into one chip, drastically reducing size, power, and complexity.

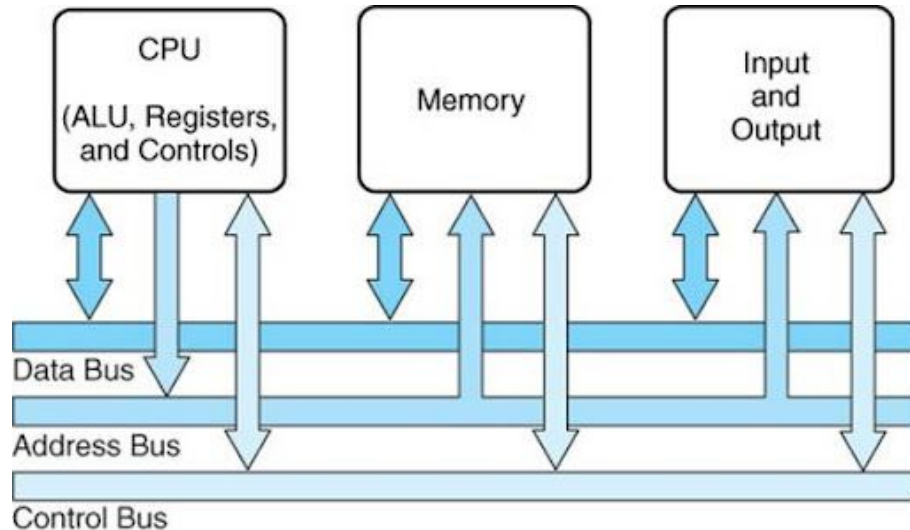
Microcontroller



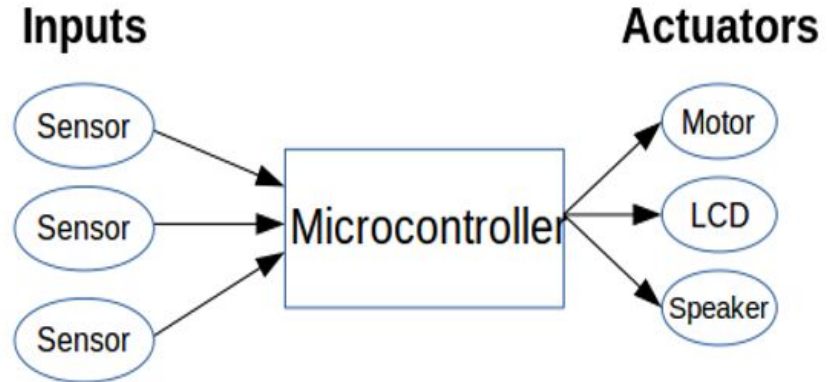
All components are connected via an internal bus and are all integrated on one chip. The modules are connected to the outside world via I/O pins.

Microprocessor vs. Microcontroller Architecture

A Microprocessor requires external connection to other components to work, for example the instruction and data memories

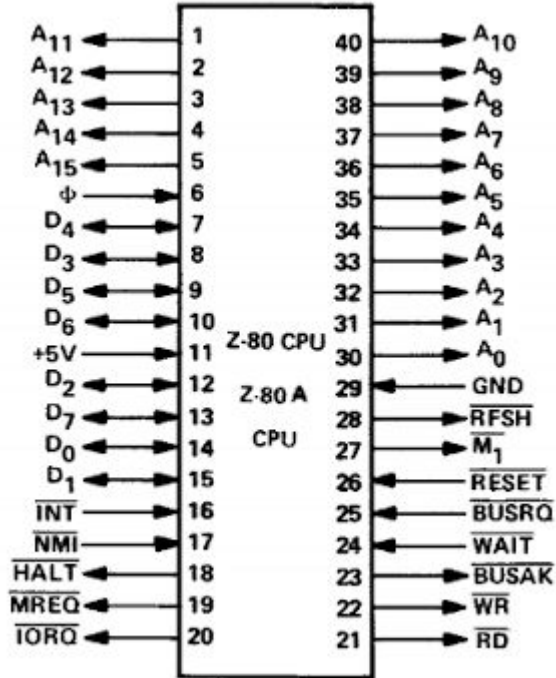


A Microcontroller embeds a reduced system inside a chip the can be directly connected to the environment with sensors/actuators

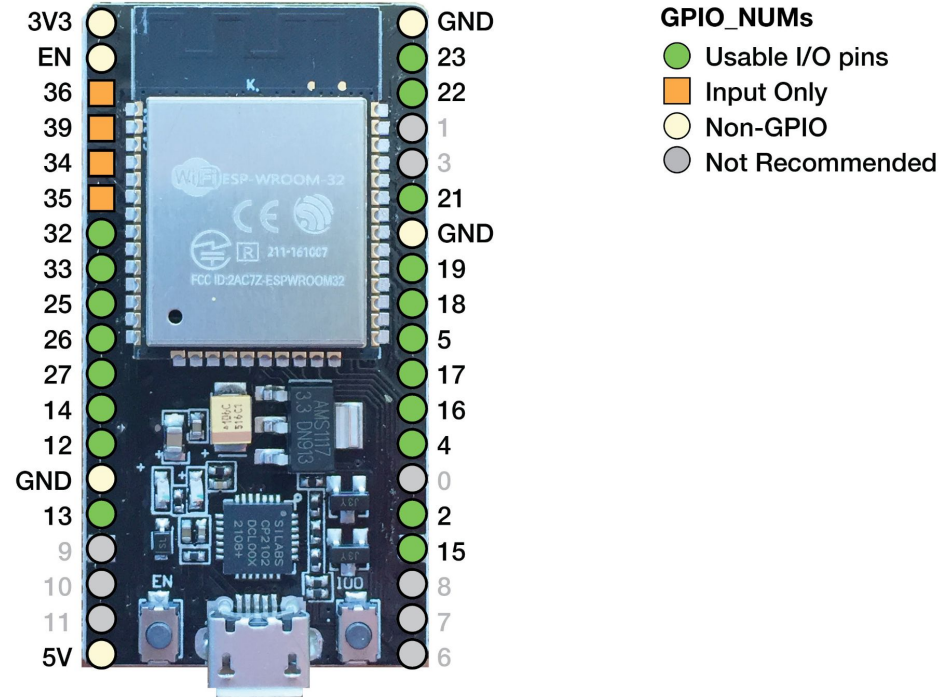


Z80: most pins are pins to memory:

- 16 bits address (A0-A15)
- 8 bits data (D0-D7)



Microcontroller: input/output pins to the external world (sensors/actuators)
Memory and other already onchip!

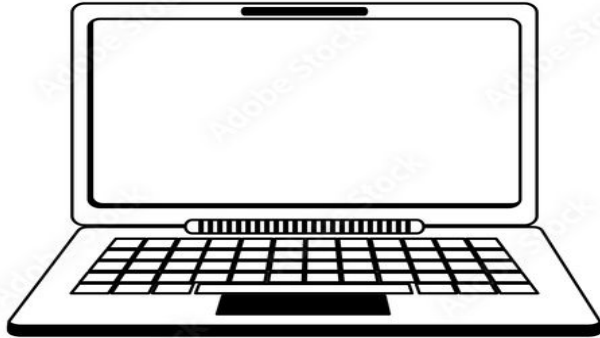


MC is Application Specific

- Design is focused on one application
 - Unlike General Purpose systems (*e.g.*, PC, laptops)
- A PC with Quad core / 4 GHz less than 1% CPU load for 99.9% of the time!
 - GP systems are inefficient, but that's ok, they should be ready to execute whatever the user wants
- Compare a € 500 computer with a similar cost videogame console (specialized)

Microcontroller

- Needs to be programmed
 - Write a program in a language on a host machine
 - Place the program in the **microcontroller memory**



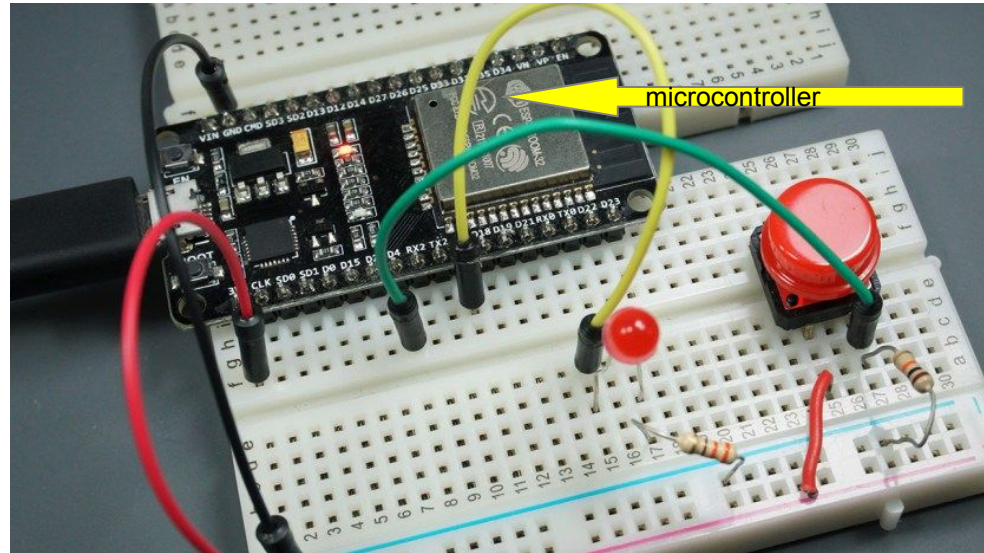
Host machine



Prototyping Boards

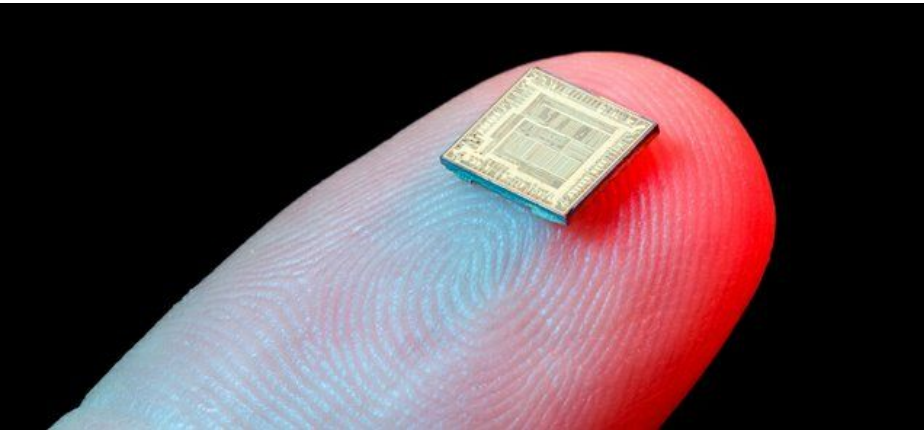
Microcontroller are usually provided into a “prototyping board” to facilitate their use:

- Cabling for power (for example, USB)
- **Serial Port:** for connecting normal computers to update program executed
- **Input/Output Pins:** to connect sensors and actuators



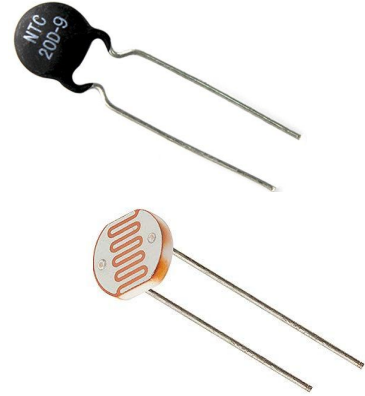
Microcontroller

- Integrated circuit that executes a program
 - Microcontroller vs. Microprocessor
 - Slower, 16 MHz – 500 MHz
 - Less memory, fewer features



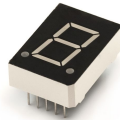
Sensors

- Receive information from the environment
- Provide simple information
 - Thermistor: reports temperature
 - Photoresistor: reports light intensity



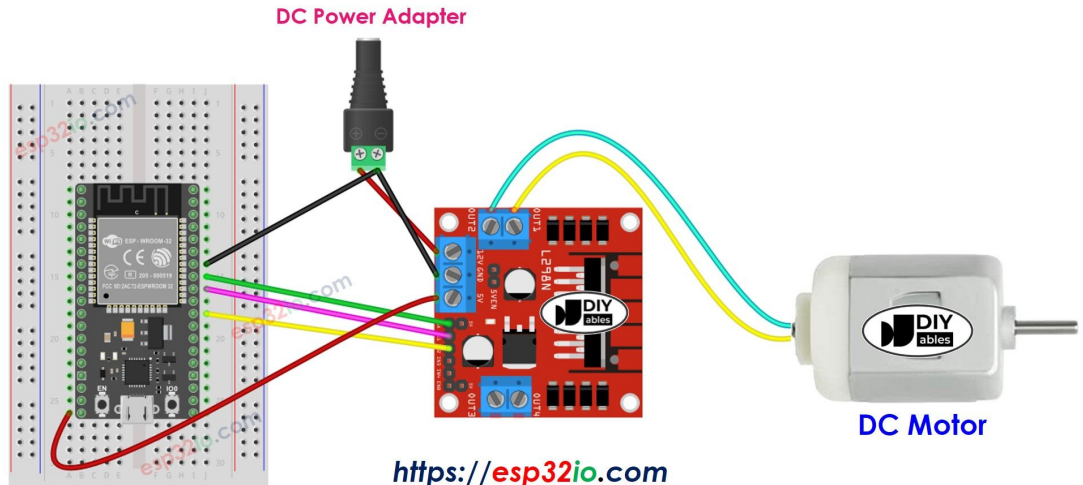
Actuators

- Cause events to occur in the environment
- Simple actuators
 - Light-Emitting Diode (LED) – small light
 - LCD Display – simple display
 - 7 segment display



Complex Actuators

- Servo motors: moves something
- Speaker
- Ethernet controller: enables network communication



Analog vs. Digital

- Analog values: continuous values, like real numbers
- Physical World consists of analog values: light, sound, temperature
- ...but inside the microcontroller, only bits can be stored! (remember, n bits $\rightarrow 2^n$ combinations)

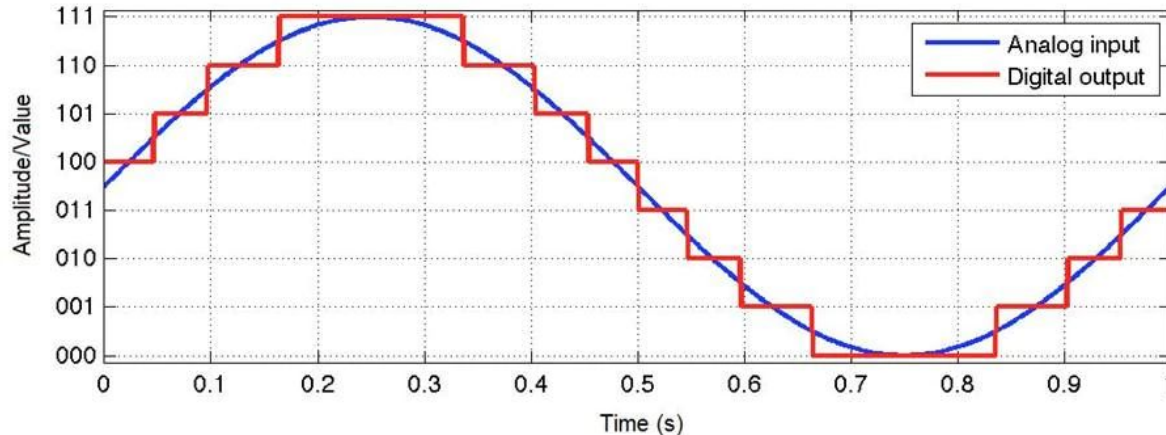
Analog to Digital Conversion

- Since physical phenomena often are analog signals, we must convert analog data to digital data
- MC provide ADC to interface with analog sensors



ADC Example

- Sound is a pressure wave over time
- A microphone converts pressure to voltage
- Voltage is sampled over time
- Each voltage level is represented as a digital number
- In this example: the real value is continuous (blue) but if we have 3 bits conversion, we must choose one of the 8 possible values



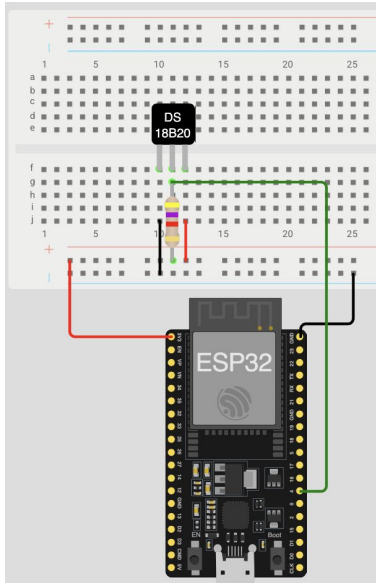
Digital to Analog Conversion

- The opposite problem: we have a value inside the microcontroller, but we want something to happen in the external world
- Example: I have a value of 3 bits, I want to use it to pilot a lamp with different intensities.
- Thus, I have 8 possible light levels, from 0 (min) to 7 (max).
- Notice: if the MC uses another size, e.g., 4 bits, there will be 16 levels of light, but 0 (min) and 15 (max) give the same as before



Hardware and Software

- IoT devices are a combination of **HW** and **SW**
- HW and SW components must be designed together



+

```
void loop() {  
    digitalWrite(13, HIGH);  
    delay(1000);  
    digitalWrite(13, LOW);  
    delay(1000);  
}
```

A Plethora of Microcontrollers

- How do we select the most appropriate one for our application?
- Look at the datasheet!
 - Many information
 - Difficult to read

Application Specific Efficiency

How to choose a Microcontroller ?

Not just *design to work* but **design to work efficiently, according to different goals:**

- Cost-critical markets (e.g., consumer electronics)
- Power-critical markets (e.g., mobile devices)
- Performance-critical markets (e.g., video games)
- Safety-critical markets (e.g., medical, military, space etc.)

Mobile Constraints

- Manufacturing cost
- Design cost
- Performance
- Power
- Reliability
- Time-to-market



Primary
constraints

Health Devices Constraints

- Manufacturing cost
- Design cost
- Performance
- Power
- Reliability
- Time-to-market

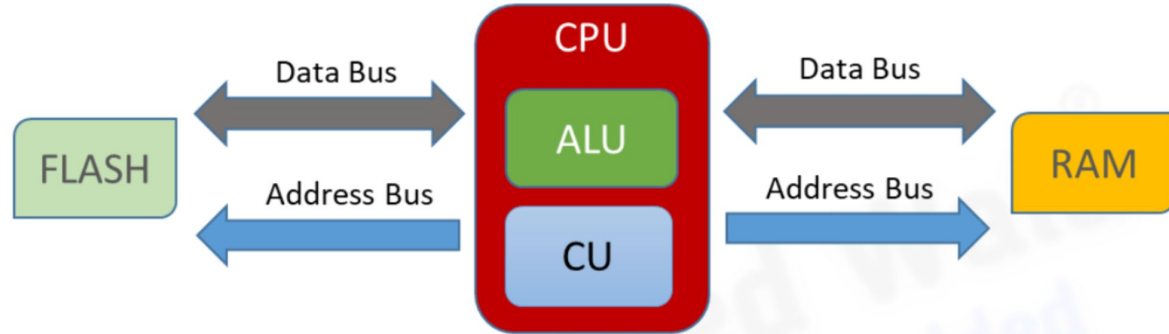


Primary
constraints

Choosing Microcontroller

- Which microcontroller to use for a given application?
- Costs are important
 - Select the **cheapest** device that **matches** the **application's needs**
 - Microcontrollers are generally **tailored for specific applications**
 - There is a wide variety of microcontrollers to choose from

Microcontroller Memory



- **Flash memory** contains the program to be executed. It maintains content even if not powered
- **RAM memory** contains variables values, that can be cancelled when restarting the program (usually a static SRAM)
- Both memory are accessed using two kind of buses:
 - Address bus: used to specify the location to be accessed
 - Data bus: used to write/read the content of the location accessed

- While SRAM is for temporary work and Flash is for the permanent program, **EEPROM (Electrically Erasable Programmable Read-Only Memory)** exists to store small bits of data that must survive a power outage but change frequently during the device's life.
- *Example: credentials, user preferences, calibration data*

Feature	SRAM (RAM)	Flash (ROM)	EEPROM
Volatile?	Yes (Lost on power off)	No (Saved)	No (Saved)
Write Unit	Single Byte	Large Blocks/Sectors	Single Byte
Speed	Extremely Fast	Fast Reads / Slow Writes	Slowest
Endurance	Infinite	~10k to 100k cycles	~100k to 1M+

Microcontroller Features

- **Flash Memory:** where to put the program
- **SRAM:** variable values used by the program
- **EEPROM:** values that should survive resets
- **I/O pins:** how many input/output connections to the external world
- **A/D:** how many pins can be used to convert analog/digital values

Controller	Flash (KB)	SRAM (Byte)	EEPROM (Byte)	I/O-Pins	A/D (Channels)
AT90C8534	8	288	512	7	8
AT90LS2323	2	128	128	3	
AT90LS2343	2	160	128	5	
AT90LS8535	8	512	512	32	8
AT90S1200	1	64		15	
AT90S2313	2	160	128	15	
ATmega128	128	4096	4096	53	8
ATmega162	16	1024	512	35	
ATmega169	16	1024	512	53	8
ATmega16	16	1024	512	32	8

MC have no or a very simplified instruction pipeline:

Low computational efforts, the main, small programs interacting with environment

Smaller MC often don't have a cache memory:

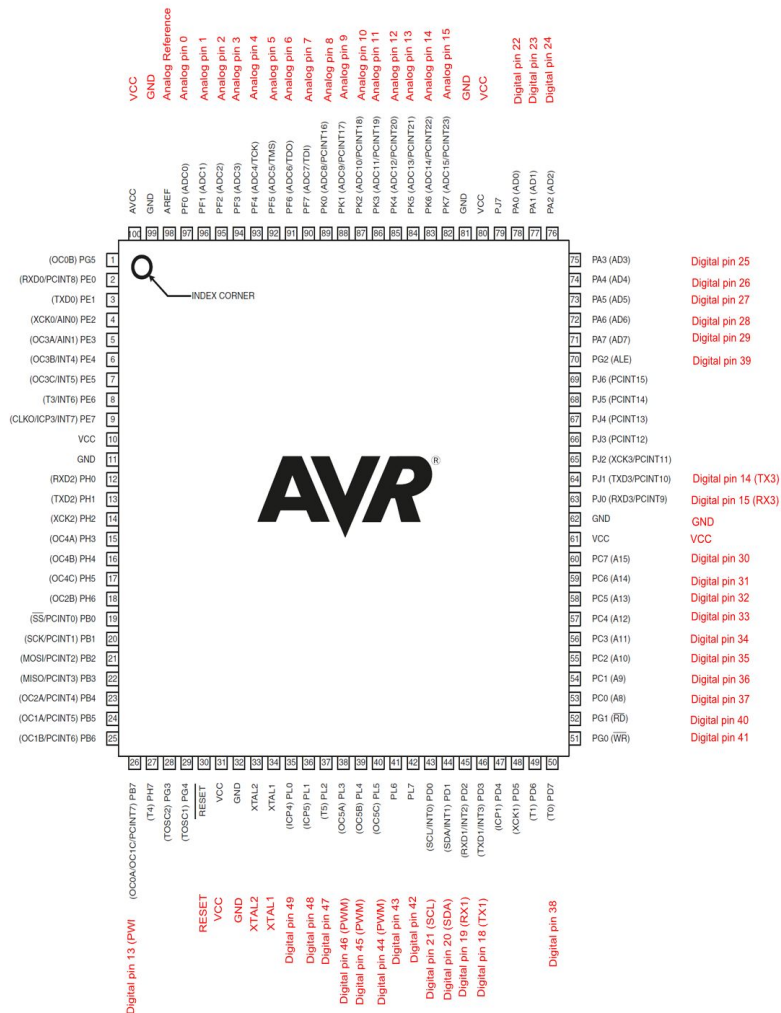
- Small memories: Think about the memory sizes shown in the previous table and what usual cache sizes are
- On-chip memory: Think about the main motivation of a cache, avoiding delay to external main RAM accesses...

Some exceptions (e.g. ESP32):

- When processor is enough fast (e.g. >100MHz) the flash may become a bottleneck -> Cache is used

- 8-bit microcontroller
- Up to 16 MHz
- 256 KB flash memory
- 4 KB EEPROM
- 8 KB SRAM
- Peripherals

- 8-bit microcontroller
- Up to 16 MHz
- 256 KB flash memory
- 4 KB EEPROM
- 8 KB SRAM
- Peripherals



Memorize each single pin for exams!

Digital pin 4 (PWM)
Digital pin 0 (RX0)
Digital pin 1 (TX0)

Digital pin 5 (PWM)
Digital pin 2 (PWM)
Digital pin 3 (PWM)

VCC
GND
Digital pin 17 (RX2)
Digital pin 16 (TX2)

Digital pin 6 (PWM)
Digital pin 7 (PWM)
Digital pin 8 (PWM)
Digital pin 9 (PWM)
Digital pin 53 (SS)
Digital pin 52 (SCK)
Digital pin 51 (MOSI)
Digital pin 50 (MISO)
Digital pin 10 (PWM)
Digital pin 11 (PWM)
Digital pin 12 (PWM)



(OC0B) PG5
(RXD0/PCINT8) PE0
(TXD0) PE1
(XCK0/AIN0) PE2
(OC3A/AIN1) PE3
(OC3B/INT4) PE4
(OC3C/INT5) PE5
(T3/INT6) PE6
(CLK0/ICP3/INT7) PE7

VCC
GND
(RXD2) PH0
(TXD2) PH1

VCC
GND
(OC4B) PH4
(OC4C) PH5
(OC2B) PH6

(SS/PCINT0) PB0
(SCK/PCINT1) PB1
(MOSI/PCINT2) PB2
(MISO/PCINT3) PB3
(OC2A/PCINT4) PB4
(OC1A/PCINT5) PB5
(OC1B/PCINT6) PB6



VCC
GND
Analog pin 0
Analog pin 1
Analog pin 2
Analog pin 3
Analog pin 4
Analog pin 5
Analog pin 6
Analog pin 7
Analog pin 8
Analog pin 9
Analog pin 10
Analog pin 11
Analog pin 12
Analog pin 13
Analog pin 14
Analog pin 15
GND
VCC
Digital pin 22
Digital pin 23
Digital pin 24

(ADC0)
(ADC1)
(ADC2)
(ADC3)
(ADC4/TK0)
(ADC5/TMS)
(ADC6/TDO)
(ADC7/ID0)
(ADC8/PCINT16)
(ADC9/PCINT17)
(ADC10/PCINT18)
(ADC11/PCINT19)
(ADC12/PCINT20)
(ADC13/PCINT21)
(ADC14/PCINT22)
(ADC15/PCINT23)

(AD0)
(AD1)
(AD2)

XTAL1
XTAL2
Digital pin 49
Digital pin 48
Digital pin 47
Digital pin 46 (PWM)
Digital pin 45 (PWM)
Digital pin 44 (PWM)
Digital pin 43
Digital pin 42
Digital pin 21 (SCL)
Digital pin 20 (SDA)
Digital pin 19 (RX1)
Digital pin 18 (TX1)

Digital pin 38

Summary

- A microcontroller is a (stripped-down) processor equipped with memory, timers, (parallel) I/O pins and other on-chip peripherals
- Integrating all elements on one chip saves space and leads to both lower manufacturing costs and shorter development times

OBSERVATION 1: Even ignoring costs, scenarios such as low power IoT devices would be impossible to realize with traditional CPU based boards

OBSERVATION 2: Some applications might still call for a Application-specific Integrated Circuit (ASIC) solution

For example: scenarios with Performance/Reliability as primary goal, with no cost or size constraint (*Google Tensor Processing Unit (TPU), Bitcoin Mining, Wearable Health Monitors, Satellite Communication Chips, Credit Card Chips*)